

Step 5 — Supervisor + Command Routing — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

state.py

```
from typing import TypedDict
from agents.research_agent import ResearchNotes
from agents.analysis_agent import Findings
from agents.reviewer_agent import ReviewVerdict

class ContentForgeState(TypedDict):
    topic: str
    research_notes: ResearchNotes
    findings: Findings
    draft: str
    review_verdict: ReviewVerdict
    revision_count: int
    completed_steps: list
    final_report: str
```

agents/supervisor.py

```
from langgraph.types import Command
from langgraph.graph import END

def supervisor_node(state):
    done = state["completed_steps"]
    if "research" not in done:
        return Command(goto="research")
    if "analysis" not in done:
        return Command(goto="analysis")
    if "writer_reviewer" not in done:
```

```

    return Command(goto="writer")
return Command(goto=END, update={"final_report": state["draft"]})

```

graph.py

```

from langgraph.graph import StateGraph, START
from state import ContentForgeState
from agents.supervisor import supervisor_node
from agents.research_agent import run_research
from agents.analysis_agent import run_analysis
from agents.writer_agent import run_writer
from agents.reviewer_agent import run_reviewer
from langgraph.types import Command

def research_node(state):
    notes = run_research(state["topic"])
    return Command(goto="supervisor", update={"research_notes": notes, "completed_st

def analysis_node(state):
    findings = run_analysis(state["research_notes"])
    return Command(goto="supervisor", update={"findings": findings, "completed_st

def writer_node(state):
    feedback = state["review_verdict"].feedback if state.get("review_verdict") el
    notes_text = "\n".join(f"- {t.label}: {'', '}.join(t.supporting_facts))" for t
    draft = run_writer(notes_text, feedback)
    return Command(goto="reviewer", update={"draft": draft, "revision_count": sta

def reviewer_node(state):
    verdict = run_reviewer(state["draft"])
    if verdict.approved or state["revision_count"] >= 3:
        return Command(goto="supervisor", update={"review_verdict": verdict, "com
    return Command(goto="writer", update={"review_verdict": verdict})

builder = StateGraph(ContentForgeState)
builder.add_node("supervisor", supervisor_node)
builder.add_node("research", research_node)
builder.add_node("analysis", analysis_node)
builder.add_node("writer", writer_node)
builder.add_node("reviewer", reviewer_node)
builder.add_edge(START, "supervisor")
graph = builder.compile()

```

main.py

```
from graph import graph

initial_state = {
    "topic": "electric bikes",
    "research_notes": None,
    "findings": None,
    "draft": None,
    "review_verdict": None,
    "revision_count": 0,
    "completed_steps": [],
    "final_report": None,
}
result = graph.invoke(initial_state)
print(result["final_report"])
```

This works — the full pipeline runs start to finish, the Supervisor routes in order, and the Writer ↔ Reviewer loop reconnects correctly. It's missing type hints and `Literal` return annotations on the node functions, and nothing stops the graph from looping forever if a bug ever breaks `completed_steps` .

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed nodes, `Literal` routing, real project layout

`state.py`

```
from typing import TypedDict, Optional
from agents.research_agent import ResearchNotes
from agents.analysis_agent import Findings
from agents.reviewer_agent import ReviewVerdict

class ContentForgeState(TypedDict):
    topic: str
    research_notes: Optional[ResearchNotes]
    findings: Optional[Findings]
    draft: Optional[str]
    review_verdict: Optional[ReviewVerdict]
    revision_count: int
    completed_steps: list[str]
    final_report: Optional[str]

def make_initial_state(topic: str) -> ContentForgeState:
    return {
        "topic": topic,
        "research_notes": None,
        "findings": None,
        "draft": None,
        "review_verdict": None,
        "revision_count": 0,
        "completed_steps": [],
        "final_report": None,
    }
```

`agents/supervisor.py`

```
from typing import Literal
from langgraph.graph import END
from langgraph.types import Command
```

```

from state import ContentForgeState

MAX_REVISION_ROUNDS = 3

def supervisor_node(
    state: ContentForgeState,
) -> Command[Literal["research", "analysis", "writer", "__end__"]]:
    done = state["completed_steps"]
    if "research" not in done:
        return Command(goto="research")
    if "analysis" not in done:
        return Command(goto="analysis")
    if "writer_reviewer" not in done:
        return Command(goto="writer")
    return Command(goto=END, update={"final_report": state["draft"]})

```

graph.py (nodes now live in their own agent files; sketch shown inline for brevity)

```

from typing import Literal
from langgraph.graph import StateGraph, START
from langgraph.types import Command
from state import ContentForgeState
from agents.supervisor import supervisor_node, MAX_REVISION_ROUNDS
from agents.research_agent import run_research
from agents.analysis_agent import run_analysis
from agents.writer_agent import run_writer
from agents.reviewer_agent import run_reviewer

def research_node(state: ContentForgeState) -> Command[Literal["supervisor"]]:
    notes = run_research(state["topic"])
    return Command(
        goto="supervisor",
        update={"research_notes": notes, "completed_steps": state["completed_steps"]}
    )

def analysis_node(state: ContentForgeState) -> Command[Literal["supervisor"]]:
    findings = run_analysis(state["research_notes"])
    return Command(
        goto="supervisor",
        update={"findings": findings, "completed_steps": state["completed_steps"]}
    )

```

```

def _findings_to_notes_text(findings) -> str:
    lines = []
    for theme in findings.themes:
        lines.append(f"{theme.label}: " + "; ".join(theme.supporting_facts))
    return "\n".join(lines)

def writer_node(state: ContentForgeState) -> Command[Literal["reviewer"]]:
    feedback = state["review_verdict"].feedback if state.get("review_verdict") else ""
    draft = run_writer(_findings_to_notes_text(state["findings"]), feedback)
    return Command(goto="reviewer", update={"draft": draft, "revision_count": state["revision_count"] + 1})

def reviewer_node(state: ContentForgeState) -> Command[Literal["writer", "supervisor"]]:
    verdict = run_reviewer(state["draft"])
    if verdict.approved or state["revision_count"] >= MAX_REVISION_ROUNDS:
        return Command(
            goto="supervisor",
            update={"review_verdict": verdict, "completed_steps": state["completed_steps"] + 1}
        )
    return Command(goto="writer", update={"review_verdict": verdict})

builder = StateGraph(ContentForgeState)
for name, node in [
    ("supervisor", supervisor_node),
    ("research", research_node),
    ("analysis", analysis_node),
    ("writer", writer_node),
    ("reviewer", reviewer_node),
]:
    builder.add_node(name, node)
builder.add_edge(START, "supervisor")
graph = builder.compile()

```

main.py

```

from graph import graph
from state import make_initial_state

def run_content_forge(topic: str) -> str:
    result = graph.invoke(make_initial_state(topic))
    return result["final_report"]

```

```
if __name__ == "__main__":  
    print(run_content_forge("electric bikes"))
```

Difference from Basic: full type hints, including `Command[Literal[...]]` return annotations on every node, which document exactly where each node is and isn't allowed to route to — genuinely useful once the graph has 5 nodes and you're debugging a routing path. `make_initial_state()` replaces a hand-typed dict literal, so `main.py` and `test_project4.py` can't drift out of sync on the state shape. `MAX_REVISION_ROUNDS` reuses Step 2's exact constant/logic, imported rather than re-typed. Still missing: any bound on the *whole graph's* hop count if `completed_steps` ever fails to update correctly, and a `supervisor_node` that's correct today but has no test forcing it to be re-checked on every call rather than assumed.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a graph-wide hop limit, surfaced as a specific error

```
# main.py
from langgraph.errors import GraphRecursionError
from graph import graph
from state import make_initial_state

class GraphStuckError(Exception):
    pass

def run_content_forge(topic: str, max_hops: int = 25) -> str:
    try:
        result = graph.invoke(make_initial_state(topic), {"recursion_limit": max_hops})
    except GraphRecursionError as e:
        raise GraphStuckError(
            f"ContentForge did not finish within {max_hops} hops for topic '{topic}'. This is likely a routing bug in the Supervisor, not a slow run."
        ) from e
    return result["final_report"]

if __name__ == "__main__":
    print(run_content_forge("electric bikes"))
```

`max_hops=25` covers a normal run generously: 1 supervisor hop before Research, 1 before Analysis, 1 before Writer, up to 3 writer ↔ reviewer round-trips (6 hops), 1 final supervisor hop to END — well under 25 even with the maximum revision rounds. If a routing bug ever causes the Supervisor to keep re-visiting a node, this turns a process you'd have to `Ctrl-C` into a clear, named `GraphStuckError` telling you exactly what to go look at.

Approach 2 — a redundancy check that's re-derived on every call, plus an explicit test for it

```
# agents/supervisor.py
from typing import Literal
```

```

from langgraph.graph import END
from langgraph.types import Command
from state import ContentForgeState

MAX_REVISION_ROUNDS = 3
STEP_ORDER = ["research", "analysis", "writer_reviewer"]
ROUTE_FOR_STEP = {"research": "research", "analysis": "analysis", "writer_reviewer": "writer_reviewer"}

def supervisor_node(
    state: ContentForgeState,
) -> Command[Literal["research", "analysis", "writer", "__end__"]]:
    done = set(state["completed_steps"])
    for step_name in STEP_ORDER:
        if step_name not in done:
            return Command(goto=ROUTE_FOR_STEP[step_name])
    return Command(goto=END, update={"final_report": state["draft"]})

```

```

# test_project4.py (excerpt)
def test_supervisor_never_reroutes_a_completed_step():
    state = make_initial_state("electric bikes")
    state["completed_steps"] = ["research", "analysis"]
    state["draft"] = "placeholder"
    command = supervisor_node(state)
    assert command.goto == "writer" # not "research" or "analysis" again

```

This is the same routing logic as Approach 1's underlying graph, made explicit and table-driven (`STEP_ORDER` / `ROUTE_FOR_STEP`) instead of a chain of `if / elif` , specifically so a test can assert the Supervisor never re-routes to a step already marked done — the exact redundant-work failure mode in this project's own problems table.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's Supervisor and graph both work correctly as long as nothing goes wrong. Approach 1 protects against the graph *itself* going wrong — a routing bug that would otherwise loop silently forever now fails fast with a specific, actionable error. Approach 2 protects against the Supervisor's *own logic* going wrong — it restructures the routing decision so it's trivially testable, and adds the test that actually proves redundant routing can't happen, rather than just hoping the `if / elif` chain stays correct as the project grows.

Which one should you actually write? Both — they protect against different failures and cost almost nothing together. The `recursion_limit` (Approach 1) is a five-minute addition with no downside: a correct graph never comes close to hitting it, and an incorrect one fails loudly instead of hanging. The table-driven Supervisor plus its test (Approach 2) is worth the extra structure specifically because this is the project's headline deliverable — "the Supervisor routes correctly" is

the first item on the project's own Checklist Before You Call This Done, and a table you can unit-test is a much stronger claim than an `if / elif` chain you're merely confident about.